TEMA 5. SISTEMAS DIGITALES

Sistemas Digitales

Tema 5

- **Introducción.**
- **Sistemas Secuenciales Asíncronos.**
  - Biestables asíncronos.
- **Sistemas Secuenciales Síncronos.**
  - Biestables síncronos
  - Análisis y síntesis de sistemas secuenciales síncronos.
    - Modelo de Mealy y Moore.
  - Bloques funcionales síncronos
    - Contadores.
    - Registros.

Introducción.

Los sistemas digitales se pueden clasificar en dos grupos, de acuerdo con su comportamiento:

$$\textit{Sistemas Digitales}\begin{cases}\cdot\ \textit{Sistemas combinacionales.}\\ \cdot\ \textit{Sistemas secuenciales}\begin{cases}\textit{asíncronos}\\ \textit{síncronos}\end{cases}\end{cases}$$

Def.1: Los sistemas combinacionales se caracterizan porque el valor lógico de sus salidas, en cualquier instante de tiempo t que se considere, sólo depende del valor lógico de sus entradas en dicho instante de tiempo t.

Def.2: Los sistemas secuenciales (sequential machine or state machine) se caracterizan porque los estados lógicos de sus salidas, en cualquier instante de tiempo t que se considere, en general, dependen de:

a) Las condiciones iniciales en las que ha comenzado a funcionar el sistema.
b) La secuencia de todas las entradas que se han aplicado al sistema (circuito) hasta dicho instante de tiempo t.
c) El estado lógico de las entradas en dicho instante t.

Ejemplos:

a) Circuito de control de un ascensor.
b) Circuito de control del cambio de canal de una T.V.

Nota: los sistemas secuenciales necesitan guardar información, mientras que los sistemas combinacionales no lo necesitan y por eso no lo hacen.

Def.3: Los sistemas secuenciales se caracterizan porque el valor lógico de sus salidas, en cualquier instante de tiempo t que se considere, dependen de:

a) El estado interno del sistema [equivale a i) y ii) en la DEF. 2].
b) El estado lógico de sus entradas [equivale a iii) en la DEF. 2].

Def.4: Se define el estado interno de un sistema secuencial como el valor representado por un conjunto de variables lógicas, denominadas variables de estado, cuyos valores en cualquier instante de tiempo t que se considere contienen la información necesaria acerca del pasado del sistema para explicar su comportamiento futuro.

Las **variables de estado** se caracterizan por lo siguiente:

a) Son variables lógicas que crea el propio sistema secuencial.
b) No representan necesariamente magnitudes físicas conocidas.
c) En la mayoría de los sistemas secuenciales se pueden definir diferentes grupos de variables de estado para representar el estado del sistema.
d) Un sistema secuencial con n variables de estado tiene como mucho 2 n estados distintos.

número de estados finito ⇒ sistemas (o máquinas) con un número finito de estados

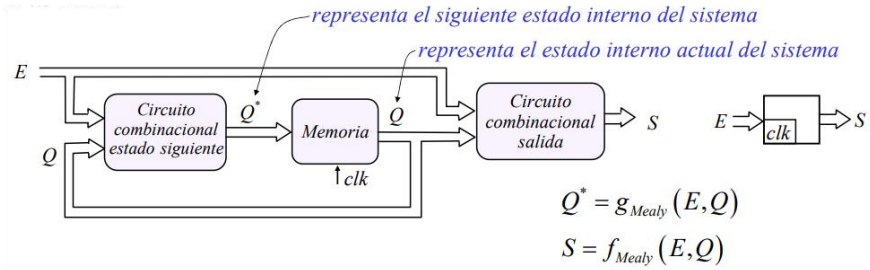
Los sistemas secuenciales se diseñan (y se analizan) a partir de los conceptos anteriores siguiendo dos estructuras o modelos distintos, denominados modelos de Mealy y de Moore.

Nota 1: Se puede demostrar que para todo sistema diseñado siguiendo el modelo de Mealy existe un modelo de Moore equivalente y viceversa. Ahora bien, aunque un diseño se pueda sustituir por otro, su funcionamiento no es exactamente el mismo.

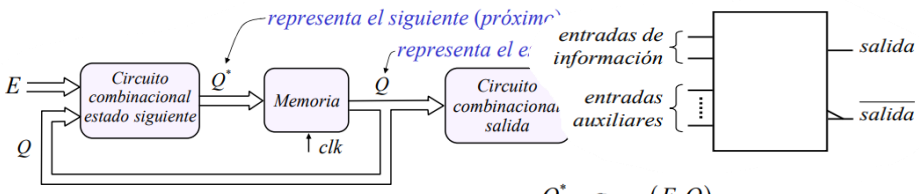
Nota 2: En general el modelo de Mealy proporciona circuitos más sencillos que el modelo de Moore. Hay sistemas cuyo comportamiento se describe mucho mejor mediante un modelo de Moore y otros mediante un modelo de Mealy.

Nota 3: Hay sistemas secuenciales que se caracterizan por tener salidas tanto de tipo Mealy como de tipo Moore… suelen ser diseños elegantes y muy eficientes.

Modelo de Mealy:



Modelo de Moore:



$$Q^* = g_{Moore}(E,Q)$$
$$S = f_{Moore}(Q)$$

E : *entradas del sistema*
S : *salidas del sistema*
Q : *estado actual*
Q^* : *estado siguiente o próximo estado*
clk : señal de sincronismo

Dispositivos de memoria **Biestables:**

Características:

a) Un biestable es un circuito digital muy sencillo que guarda (memoriza) 1 bit de información.

b) Básicamente, un biestable posee: 1 salida (Q), 1 ó 2 entradas de información (según el tipo de biestable que se considere) y, en algunos casos, entradas auxiliares.

c) Las entradas de información sirven para indicarle al biestable el valor lógico que debe memorizar (0 ó 1).

d) La salida proporciona en todo momento el valor guardado por el biestable.

e) Puede tener entradas auxiliares que permiten controlar su funcionamiento.

f) Un biestable constituye por si solo un sistema secuencial $\Rightarrow$ los biestables se pueden clasificar en asíncronos y síncronos.

Una posible clasificación de los biestables es la siguiente:

Biestables
- *Asíncronos (Latches)*
  - *Sin entrada de habilitación*: *SR* (*D*, *JK*, *T*)
  - *Con entrada de habilitación*: *D* (*SR*, *JK*, *T*)
- *Síncronos (Flip-flops)*
  - *Disparados por flanco (edge triggered)*: *D*, *JK*, *T*, (*SR*, *PQ*)

A nivel funcional, el comportamiento de un biestable se puede definir mediante:

a) Una ecuación característica: proporciona el nuevo valor a guardar (Q*) en función del valor de las entradas y del valor guardado actualmente.

$$Q^* = f(E,Q)$$

b) Una tabla de transición de estados.

c) Un diagrama de flujo o diagrama de estados.

d) Un cronograma.

Nomenclatura: $Q^* \equiv Q_{t+1} \equiv Q_{t+T} \equiv Q_{t+\Delta t} \equiv Q_t^*$

Biestables asíncronos: LATCHES:

Se caracterizan porque obedecen en todo momento a sus entradas de información.

Latch S-R:

a) Es el más sencillo de todos los biestables.

b) Se utiliza como punto de partida para diseñar cualquier otro tipo de biestable.

c) Tiene dos entradas (de información):

a. S ≡ set → sirve para indicarle al biestable (latch) que guarde un 1.

b. R ≡ reset → sirve para indicarle al biestable (latch) que guarde un 0.

Nomenclatura para biestables:

Q : representa el valor actual o presente en la salida de un biestable. Dicho de otra forma, representa el valor de la salida de un biestable antes de que ésta cambie de valor debido a un cambio de valor en una de sus entradas.

Q* : representa el siguiente valor que tomará la salida Q de un biestable. Es decir, es el valor que pasará a tener la salida de un biestable como consecuencia de un cambio de valor de una entrada (≡ es el siguiente valor que guardará el biestable).

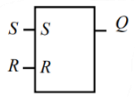
Nomenclatura para variables de estado:

Q : representa el valor actual o presente de una variable de estado. Dicho de otra forma, representa el valor de una variable de estado antes de que el sistema cambie de estado.

Q* : representa el siguiente valor que tomará una variable de estado. Es decir, es el valor que tomará la variable de estado al cambiar el sistema a un nuevo estado.

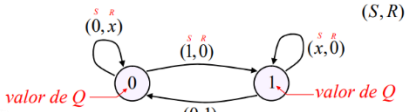
Nomenclatura: $Q_t \equiv Q$ $\quad$ $Q_{t+T} \equiv Q_{t+1} \equiv Q^*$

Definición de su comportamiento:



valor de la salida en dos instantes de tiempo distintos

S	R	Q	Q^*	
0	0	0	0	→ no hagas nada, quédate como estás $(Q^* = Q)$.
0	0	1	1	
0	1	0	0	→ ponte a memorizar un cero $(Q^* = 0)$.
0	1	1	0	
1	0	0	1	→ ponte a memorizar un uno $(Q^* = 1)$.
1	0	1	1	
1	1	0	x	→ orden prohibida
1	1	1	x	



Biestables síncronos: Flip-Flops.

Se caracterizan por lo siguiente:

a) Básicamente, tienen dos tipos de entradas:
   a. entradas de información (1 ó 2 entradas).
   b. entrada de sincronismo (1 entrada).

b) La entrada de sincronismo es la encargada de habilitar las entradas de información.

c) Las entradas de información están habilitadas durante intervalos de tiempo muy pequeños en torno a los flancos de sincronismo.

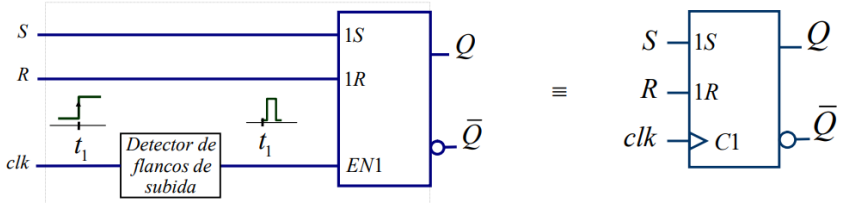
d) Algunos flip-flops tienen entradas auxiliares (asíncronas). La acción de una entrada asíncrona siempre prevalece sobre la acción de una entrada síncrona.

e) Los flip-flops se pueden clasificar en dos grupos:
   a. Biestables master-slave (¡¡¡obsoletos!!!).
   b. Biestables disparados por flanco (edge triggered).

Biestables sincronizados por flanco (edge triggered)

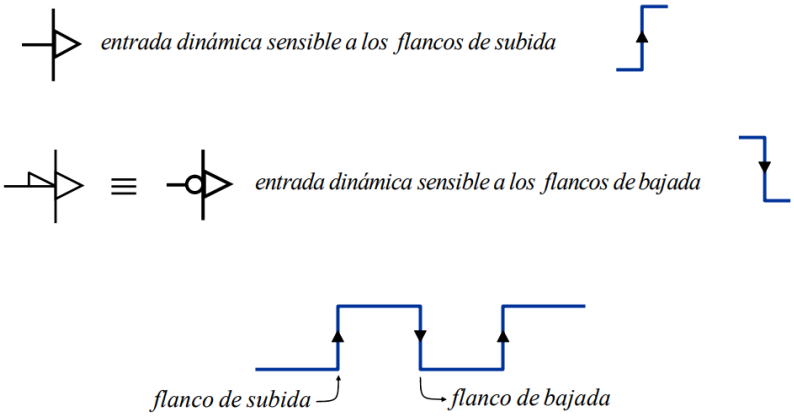
Se caracterizan por lo siguiente:

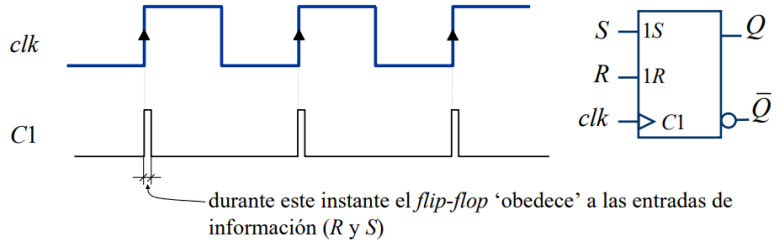
a) Se comportan como biestables asíncronos (latches) con una entrada para habilitar las entradas de información.

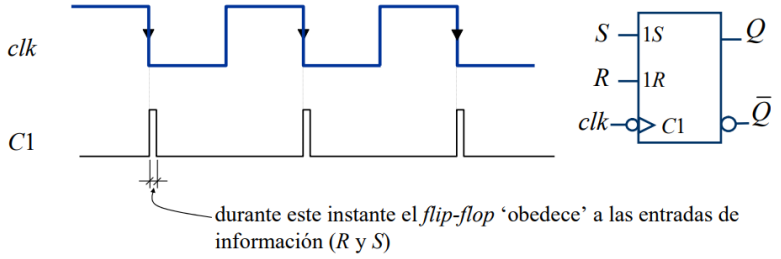


Nota: mientras el estado lógico (interno) de EN1 valga 0 o el valor de C1 valga 0, el biestable ignora las entradas de información.

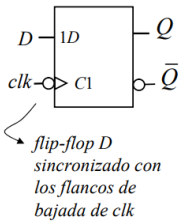
b) Las entradas de información están habilitadas durante intervalos de tiempo muy pequeños. Dichos intervalos están centrados en torno a los flancos de subida o bien de bajada que describe la señal de sincronismo (≡ señal de reloj ≡ clk).







Flip-flop D disparado por flanco:



Tiene 1 entrada de información (D) y 1 entrada de sincronismo (clk).

Definición de su comportamiento:

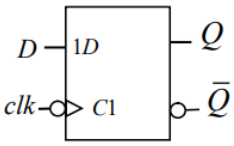
valores antes de que la señal clk describa un flanco de bajada

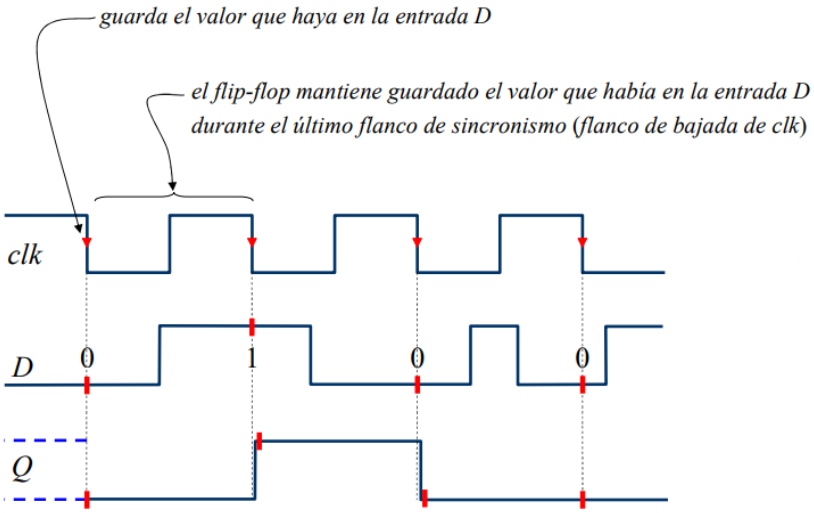
valor guardado por el flip flop después de que la señal clk describa un flanco de bajada (es igual al valor que tiene la entrada D justo antes de que clk describa el flanco de bajada)

clk	D	Q	Q*	
↓	0	0	0	*memoriza un cero* $(Q^* = D = 0)$.
↓	0	1	0	
↓	1	0	1	*memoriza un uno* $(Q^* = D = 1)$.
↓	1	1	1	

esta tabla sólo es cierta durante los flancos de bajada de la señal de sincronismo (clk)

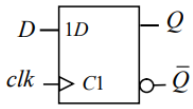
Flip-flop D sincronizado con los flancos de bajada de clk:

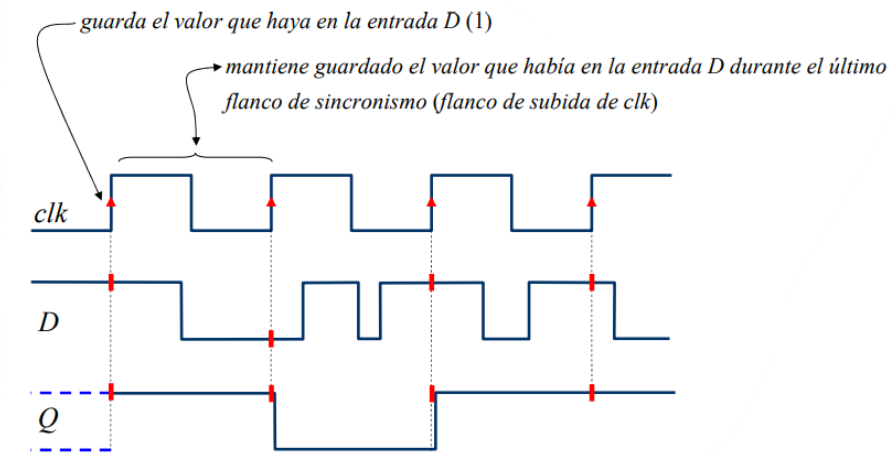




Nota: en la práctica, la salida (Q) siempre actualiza su valor con un pequeño retraso con respecto al flanco de sincronismo.

Flip-flop D sincronizado con los flancos de subida de clk:

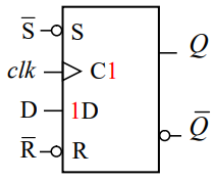




Nota: en la práctica, la salida (Q) siempre actualiza su valor con un pequeño retraso con respecto al flanco de sincronismo.

Circuito 74xxx74:

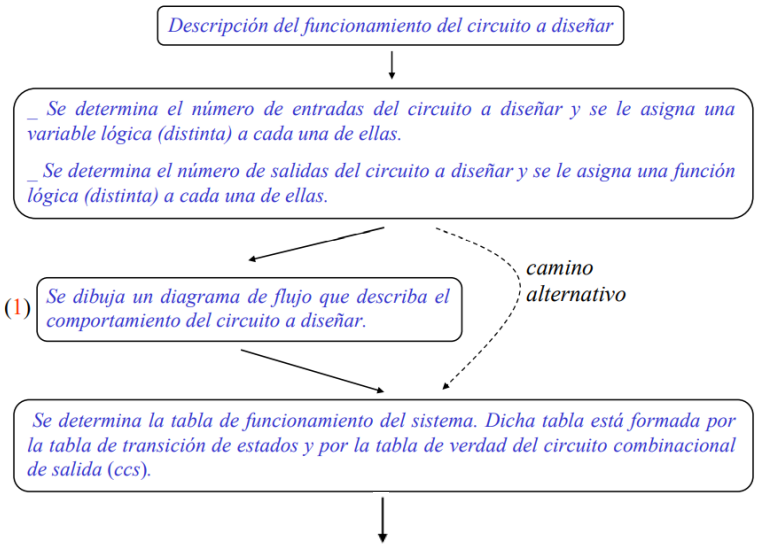
clk	$\bar{S}$	$\bar{R}$	D	Q	Q^*	
x	0	0	x	x	*prohibida*	*funcionamiento asíncrono*
x	0	1	x	x	1	
x	1	0	x	x	0	
↑	1	1	0	x	0	*funcionamiento síncrono*
↑	1	1	1	x	1	



Las entradas asíncronas siempre prevalecen sobre las entradas síncronas.

1) Entradas de información asíncronas: $\bar{S}$, $\bar{R}$
2) Entada de información síncrona: D.

Procedimiento general de diseño de un sistema secuencial síncrono.



Se minimiza el número de estados del modelo planteado (opcional)

Se codifican los estados del modelo propuesto, asignándole un número binario distinto a cada uno. El número de dígitos de los 'nombres (≡ números)' asignados a los distintos estados establece el número de variables de estado que se necesitan para representar el estado (interno) del sistema.

Cada vez que el sistema se encuentre en un determinado estado, las variables de estado (interno) tomarán el valor (número) asignado a dicho estado (interno).

Se elige el tipo de flip-flops que se van a utilizar en el bloque de memoria para guardar el valor de las variables de estado ≡ valor del estado interno. El valor de cada variable de estado se guardará en 1 flip-flop.

Se determina la tabla de excitación de los flip-flops elegidos.

Se determina la tabla de verdad del circuito combinacional del estado siguiente (cces)

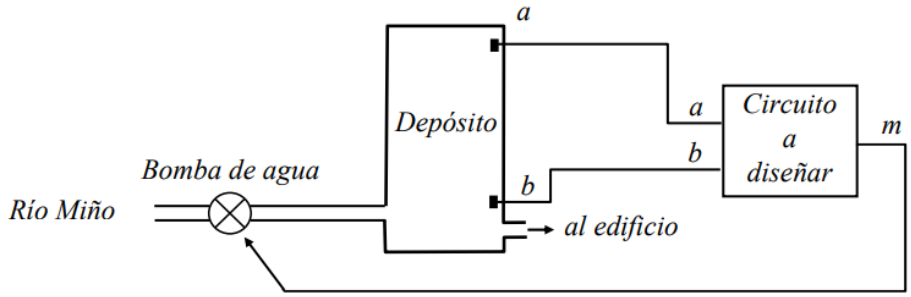
Se determinan las expresiones más sencillas de las funciones que describen el comportamiento a nivel lógico de los circuitos combinacionales del estado siguiente y de salida.

Dibujar un esquema del circuito.

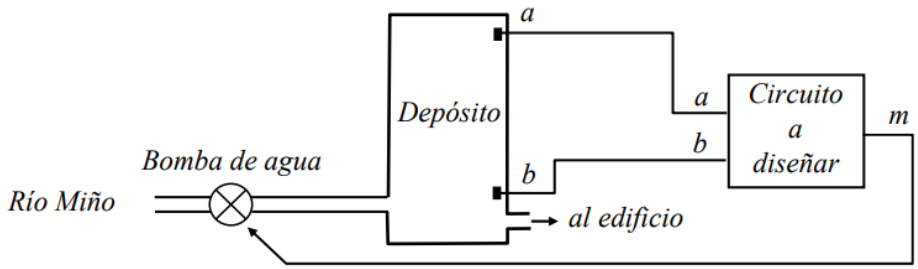
Simular el funcionamiento del circuito diseñado (opcional). Si el circuito diseñado no funciona de acuerdo con las especificaciones se vuelve al punto (1)

Construcción de un prototipo. Si su funcionamiento no cumple las especificaciones se vuelve al punto (1)

Ejemplo I de obtención de un diagrama de flujo (modelo de Moore):



Se trata de diseñar un sistema que controle el funcionamiento de la bomba (de agua), de modo que el nivel del agua en el interior del deposito esté siempre comprendido entre los niveles establecidos por la posición de los sensores a y b.



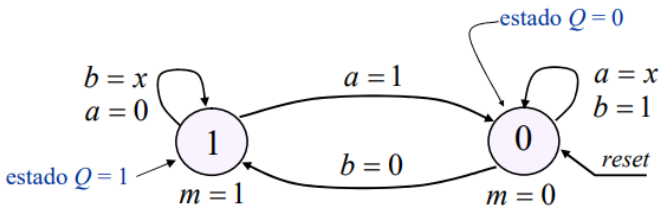
El circuito (sistema secuencial) a diseñar tiene 2 entradas (a y b) y 1 salida (m). Las variables lógicas asignadas a las entradas y la función lógica asignada a la salida se definen de la siguiente manera:

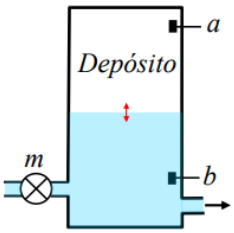
$$a = \begin{cases} 1 & \text{si el nivel del agua supera la posición del sensor (a)} \\ 0 & \text{si el nivel del agua es inferior a la posición del sensor (a)} \end{cases}$$

$$b = \begin{cases} 1 & \text{si el nivel del agua supera la posición del sensor (b)} \\ 0 & \text{si el nivel del agua es inferior a la posición del sensor (b)} \end{cases}$$

$$m = \begin{cases} 1 & \text{el motor bombea agua al depósito} \\ 0 & \text{el motor está parado} \end{cases}$$

Diagrama de flujo (modelo de Moore):





En el estado Q = 0, la bomba de agua está parada (m = 0) y el depósito se está vaciando. El sistema se mantiene en el estado Q = 0 mientras el nivel del agua sea superior al nivel en el que se encuentra el sensor b. Cuando el nivel del agua desciende por debajo de la posición del sensor b (b = 0), el sistema pasa al estado Q = 1 en el que se bombea agua al interior del deposito (m = 1). El sistema se mantiene en el estado Q

= 1 hasta que el nivel del agua en el interior del depósito alcanza el nivel en el que se encuentra el sensor a (a = 1), en cuyo momento el sistema pasa al estado Q = 0, en el que la bomba de agua está parada (m = 0).

Ejemplo II de obtención de un diagrama de flujo (modelo de Moore):

En este caso se trata de diseñar un sistema de control que haga que cada vez que se presione un pulsador α se encienda una lámpara, en el caso de que esté apagada, o se apague en el caso de que esté encendida. "La lámpara cambia de estado (encendida/apagada) cada vez que se deja de presionar el pulsador (botón) α".

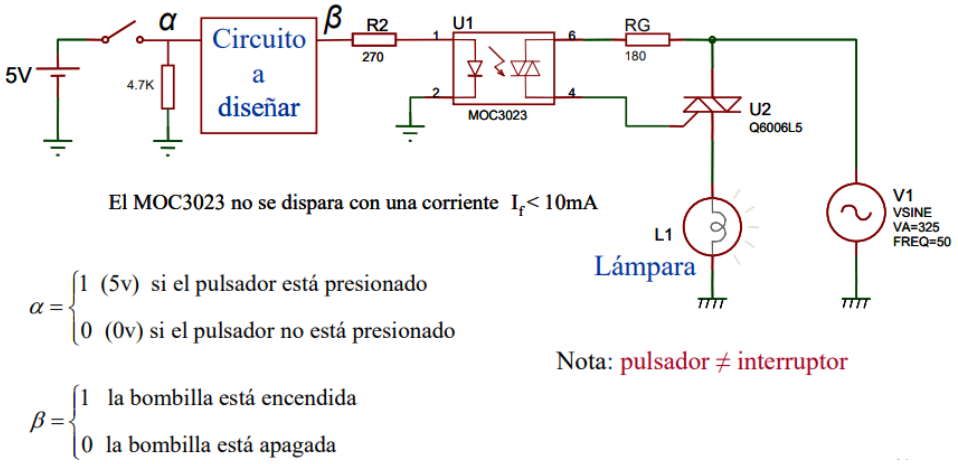
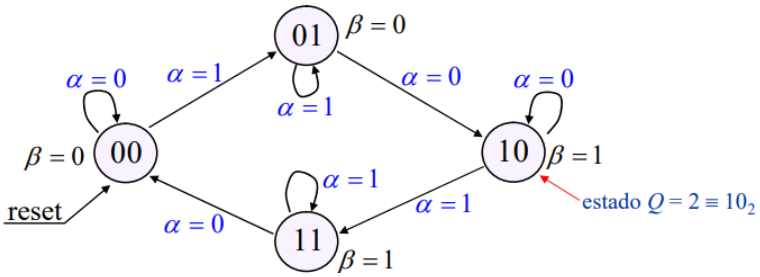


Diagrama de flujo (modelo de Moore):

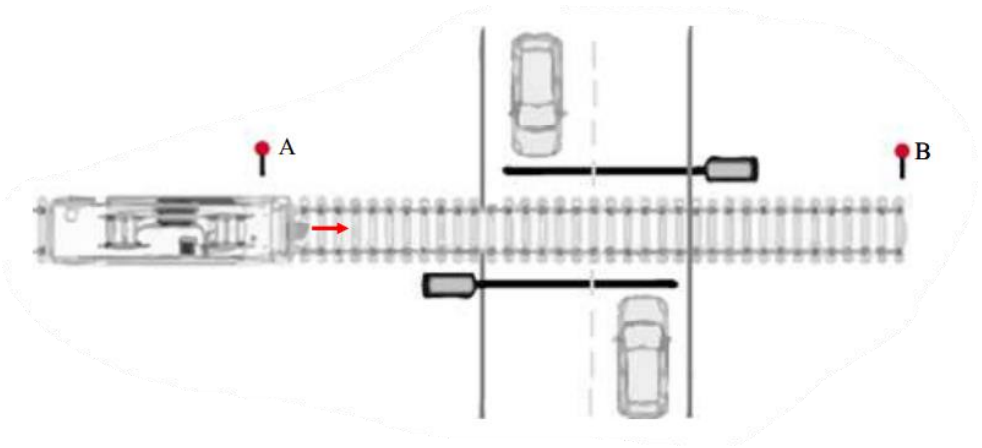


En el estado 0, la bombilla está apagada (β = 0) y el pulsador no está presionado (α = 0). Estando en este estado, si se presiona el pulsador (α = 1), el sistema pasa al estado 1, en el que permanece hasta que se deja de presionar el pulsador (α = 0), en cuyo momento el sistema pasa al estado 2. En dicho estado la bombilla está encendida (β = 1) y el pulsador no está presionado (α = 0). Estando en el estado 2, si se presiona el pulsador el sistema pasa al estado 3, en el que permanece hasta que se deja de presionar el pulsador. En cuyo momento el sistema pasa al estado 0, en el que la bombilla permanece apagada.

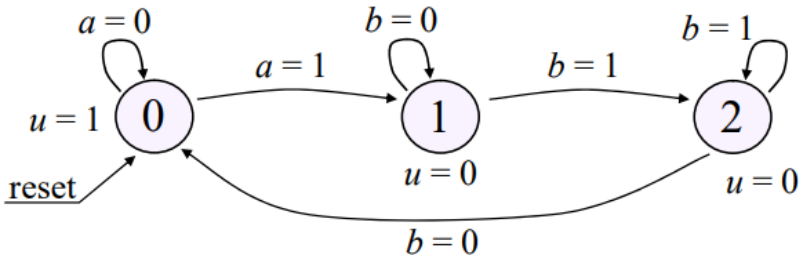
Ejemplo III de obtención de un diagrama de flujo (modelo de Moore):

En este caso se trata de determinar un diagrama de estados que modele el sistema de control de las barreras de un paso a nivel. A la hora de determinar el diagrama de estados hay que tener en cuenta lo siguiente:

a) Hay dos sensores (A y B) sobre la vía del tren, situados cada uno a un lado de la carretera que atraviesa la vía.
b) Cada sensor (A, B) tiene una salida binaria en la que hay un 1 lógico siempre que un tren esté pasando por delante de dicho sensor y hay un 0 lógico siempre que no hay un tren pasando por delante del mismo.
c) La posición de las barreras se controla mediante una señal binaria u que cumple lo siguiente:
    a. u = 1 → las barreras están subidas.
    b. u = 0 → las barreras están bajadas.
d) Las barreras deben estar bajadas (u = 0) siempre que haya al menos una parte de un tren en el trayecto de la vía férrea delimitado por los sensores A y B. Las barreras no se pueden subir (permitiendo la circulación de los coches) hasta que todo el tren se encuentre fuera del tramo comprendido entre las posiciones de los sensores A y B.
e) La velocidad del tren está limitada en el tramo comprendido entre las posiciones en las que se encuentran los sensores (A y B) de modo que, cuando el tren se aproxima al paso a nivel, el tiempo que este tarda en recorrer la distancia comprendida entre el sensor A y el paso a nivel es mucho mayor que el tiempo que tardan en bajarse las barreras.
f) La longitud de un tren (locomotora + vagones) puede ser mayor o menor que la distancia que separa a los sensores A y B.
g) Los trenes sólo pueden circular en un sentido por la vía férrea considerada (de izquierda a derecha en el siguiente esquema).

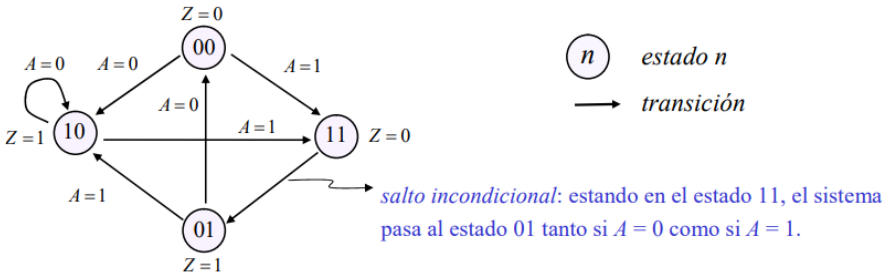


Un diagrama de estados (modelo de Moore):

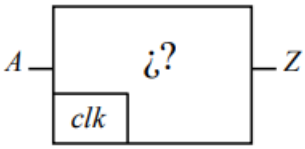


- **Estado 0:** las barreras están subidas (no hay un tren circulando por las vías en el tramo comprendido entre las posiciones de los sensores A y B).
- **Estado 1:** un tren está pasando por delante del sensor A (barreras bajadas).
- **Estado 2:** un tren está pasando por delante del sensor B (barreras bajadas).

Ejemplo de implementación de un diagrama de flujo (modelo de Moore):

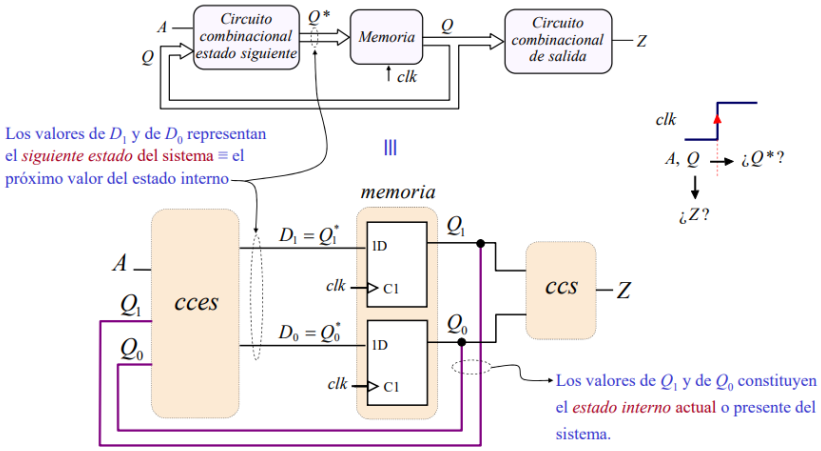


Del diagrama de flujo se deduce lo siguiente:



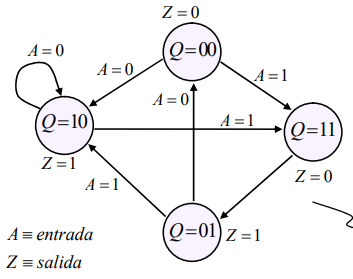
- tiene una única entrada: A.
- tiene una única salida: Z.
- tiene 4 estados, los cuales se han codificado asignándole a cada uno un número binario de 2 dígitos. De acuerdo con esto, el sistema tendrá 2 variables de estado Q(Q1 , Q0) y, por lo tanto, el bloque de memoria estará formado por 2 flip-flops (en este caso se van a utilizar flip-flops de tipo D para guardar los valores de las variables de estado).

El diagrama de bloques del circuito (sistema secuencial) a diseñar es el siguiente:



Pregunta: ¿Por qué el sistema sólo puede cambiar de estado durante los flancos de subida de la señal clk?

Para poder diseñar los circuitos combinacionales que proporcionan el estado siguiente (cces) y la salida (ccs) hay que determinar la tabla de transición de estados y la tabla de verdad del circuito combinacional de salida a partir del diagrama de flujo o de estados:



Q_1	Q_0	Z
0	0	0
0	1	1
1	0	1
1	1	0

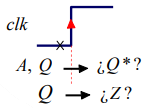
tabla de verdad del ccs

estado actual (*dura hasta el próximo flanco de sincronismo*) ≡ valores antes de un flanco de subida de *clk*.

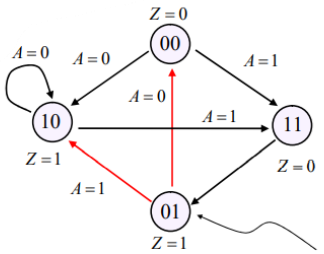
Q			Q^*	
Q_1	Q_0	A	Q_1^*	Q_0^*
0	0	0	1	0
0	0	1	1	1
0	1	0	0	0
0	1	1	1	0
1	0	0	1	0
1	0	1	1	1
1	1	0	0	1
1	1	1	0	1

estado siguiente (*será el estado en el que estará el sistema a partir del próximo flanco de sincronismo*) ≡ valores después de un flanco de subida de *clk*.

tabla de transición de estados



Ejemplo:



Q_1	Q_0	A	Q_1^*	Q_0^*
0	0	0	1	0
0	0	1	1	1
0	1	0	0	0
0	1	1	1	0
1	0	0	1	0
1	0	1	1	1
1	1	0	0	1
1	1	1	0	1

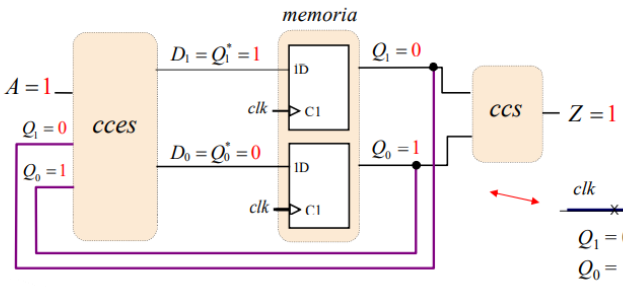
Siempre que el sistema esté en el estado $Q_1\,Q_0 = 01$, se cumple que:

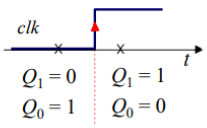
_ la salida es $Z = 1$

_ si $A = 0$, a partir del siguiente flanco de subida de *clk*, el estado del sistema será el 00

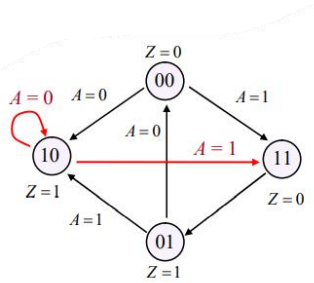
_ si $A = 1$, a partir del siguiente flanco de subida de *clk*, el estado del sistema será el 10

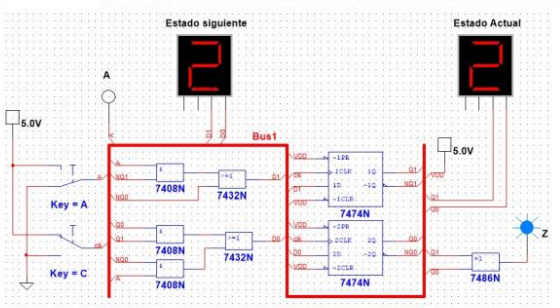
Q_1	Q_0	Z
0	0	0
0	1	1
1	0	1
1	1	0



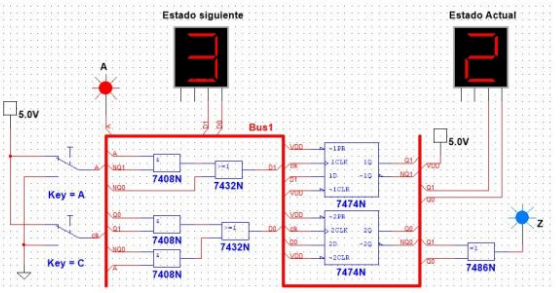


Ejemplo con el simulador:

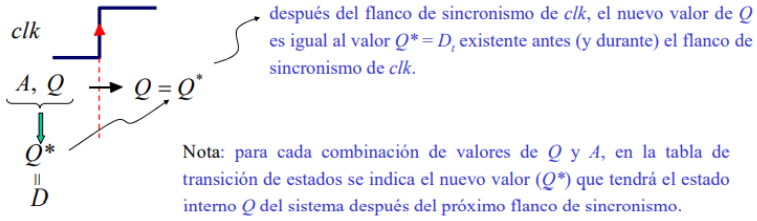




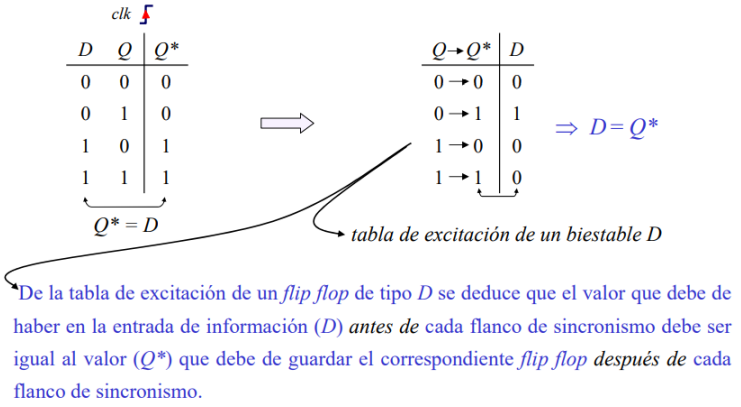
Q_1	Q_0	A	Q_1^*	Q_0^*
0	0	0	1	0
0	0	1	1	1
0	1	0	0	0
0	1	1	1	0
1	0	0	1	0
1	0	1	1	1
1	1	0	0	1
1	1	1	0	1



La tabla de transición de estados indica el nuevo valor (Q*) que tendrá el estado interno después del próximo (siguiente) flanco de sincronismo, en función de los valores actuales del estado (Q) y de la entrada (A). Para que se produzca el cambio de estado correcto, durante los flancos de sincronismo de la señal clk es necesario que los valores D_1 y D_0 presentes en las entradas de los flip-flops, tanto antes como durante un flanco de sincronismo, correspondan a los valores Q_1 y Q_0 que deberá tener el estado interno del sistema después de que ocurra dicho flanco de sincronismo (y que en la tabla de transición de estados se representan por y). Los valores de D_1 y de D_0 los genera el circuito combinacional del estado siguiente a partir del estado actual (Q_1 y Q_0) del sistema y del valor actual de la entrada (A).



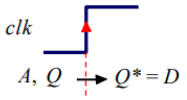
Para determinar el valor D (D_1 , D_0) que debe haber en las entradas de información de los flip flops resulta útil determinar previamente la tabla de excitación de los biestables que se vayan a utilizar en el bloque de memoria.



A partir de la tabla de transición de estados y de la tabla de excitación de un flip flop D se deduce la siguiente tabla de verdad de las funciones D_1 (Q_1 ,Q_0 ,A) y D_0 (Q_1 ,Q_0 ,A) a implementar por el circuito combinacional de entrada (cce):

Q_1	Q_0	A	Q_1^*	Q_0^*	D_1	D_0
0	0	0	1	0	1	0
0	0	1	1	1	1	1
0	1	0	0	0	0	0
0	1	1	1	0	1	0
1	0	0	1	0	1	0
1	0	1	1	1	1	1
1	1	0	0	1	0	1
1	1	1	0	1	0	1

$Q \rightarrow Q^*$	D
$0 \rightarrow 0$	0
$0 \rightarrow 1$	1
$1 \rightarrow 0$	0
$1 \rightarrow 1$	0



Nota: en el caso de utilizar flip flops D para guardar el valor de las variables de estado del sistema, se cumple que: D_1 (Q_1 , Q_0 , A) = y D_0 (Q_1 , Q_0 , A) = Q_0*

A partir de las tablas de verdad de los circuitos combinacionales del estado siguiente (cces) y de salida (ccs), se deducen las siguientes expresiones lógicas:

$$D_1(Q_1,Q_0,A)=\sum\nolimits_3(0,1,3,4,5)$$

$$D_0(Q_1,Q_0,A)=\sum\nolimits_3(1,5,6,7)$$

$$Z(Q_1,Q_0)=\sum\nolimits_2(1,2)$$

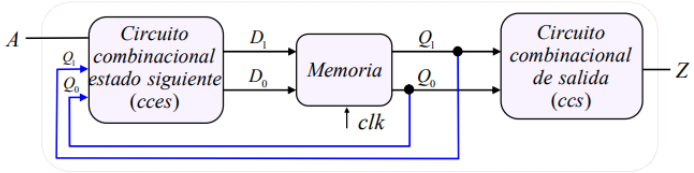
Simplificando por Karnaugh-Veitch las funciones anteriores se obtiene lo siguiente:

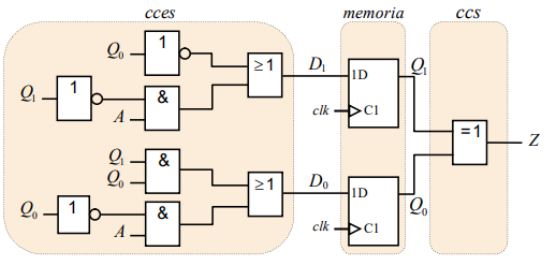
$$D_1(Q_1,Q_0,A)=\bar{Q}_1A+\bar{Q}_0$$

$$D_0(Q_1,Q_0,A)=\bar{Q}_0A+Q_1Q_0$$

$$Z(Q_1,Q_0)=\bar{Q}_1Q_0+Q_1\bar{Q}_0=Q_1\oplus Q_0$$

A partir de las expresiones anteriores se deduce el siguiente circuito:





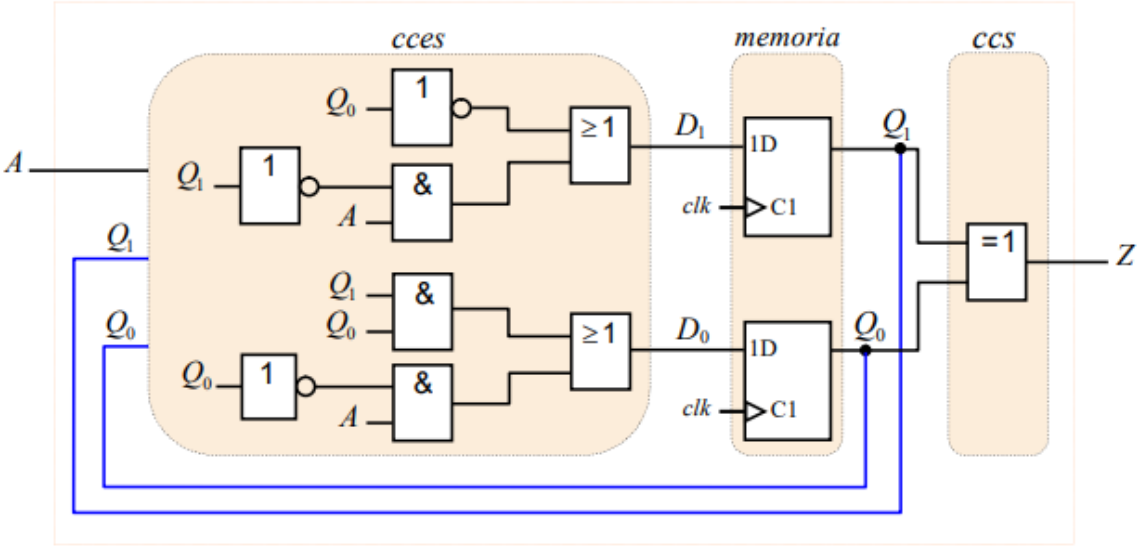
$$D_1(Q_1,Q_0,A)=\bar{Q}_1A+\bar{Q}_0$$

$$D_0(Q_1,Q_0,A)=\bar{Q}_0A+Q_1Q_0$$

$$Z(Q_1,Q_0)=Q_1\oplus Q_0$$

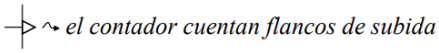
* *clk* es una señal auxiliar que sincroniza el funcionamiento de los *flip - flops*

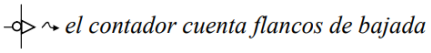
El circuito anterior, indicando las realimentaciones de las variables de estado:



5.3.2: Bloques funcionales secuenciales (síncronos)

Def.1: Un contador es un circuito secuencial síncrono que cuenta en un código binario los flancos de subida o bien de bajada que presenta la señal que se aplica a su entrada dinámica.



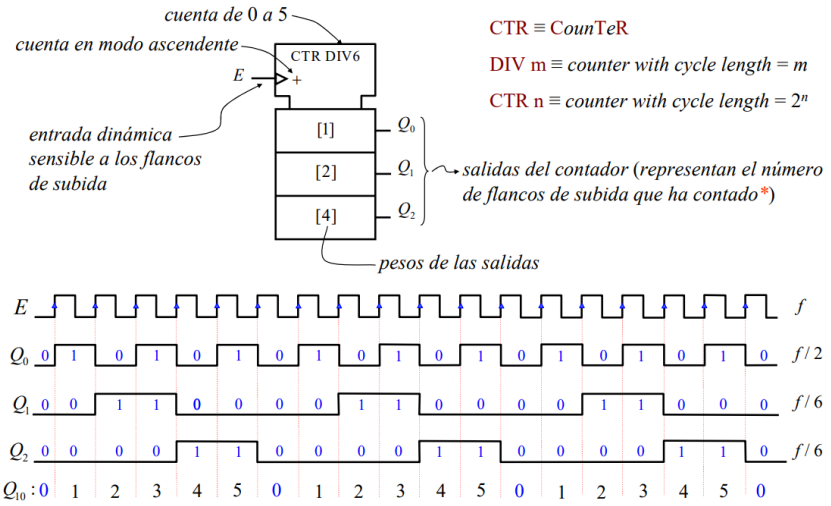


Nota 1: contar los flancos de subida o bien de bajada de una señal binaria equivale a contar sus pulsos.

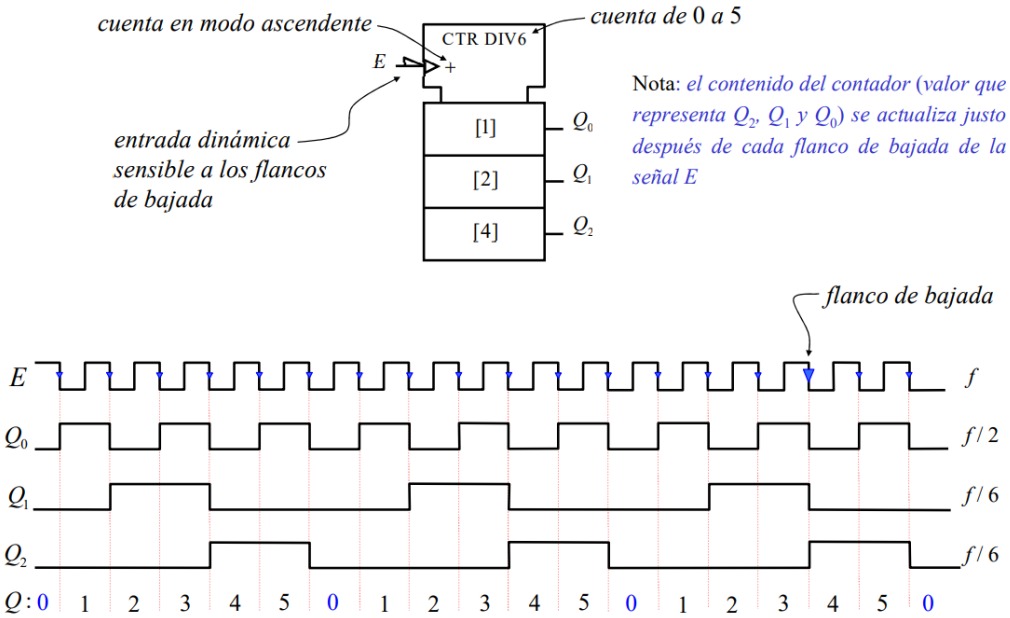
Nota 2: el número máximo de flancos que puede contar un contador es finito, ya que tiene un número finito de salidas. Cuando un contador ha contado el número máximo de flancos que puede contar, con el siguiente flanco que cuente reinicia el contaje (se produce un desbordamiento ≡ an overflow).

Nota 3: salvo que se indique lo contrario, se debe suponer que un contador cuenta en binario natural.

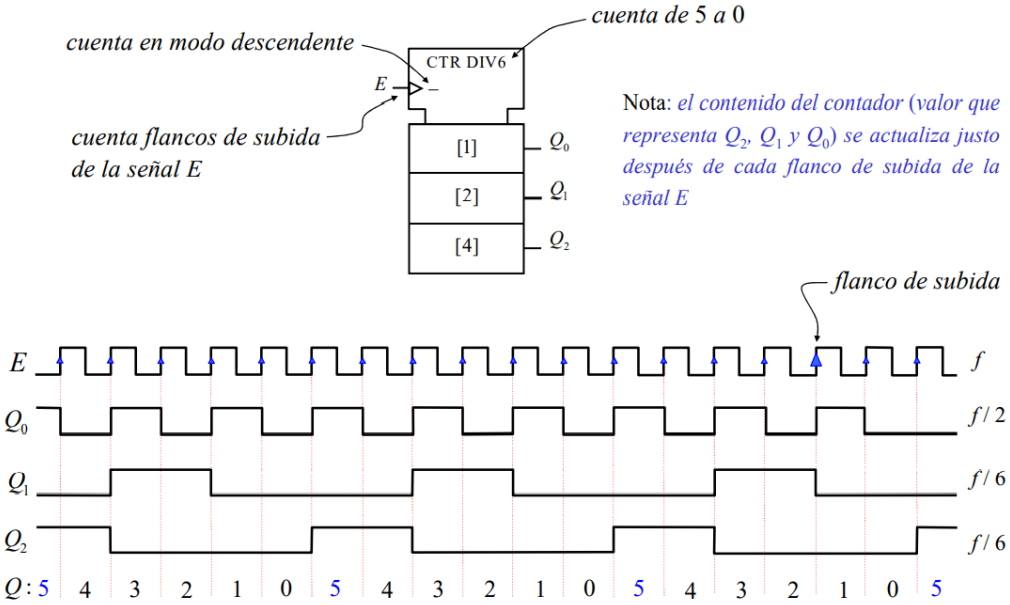
Ejemplo: contador de los flancos de subida de la señal E, en modo ascendente (+).



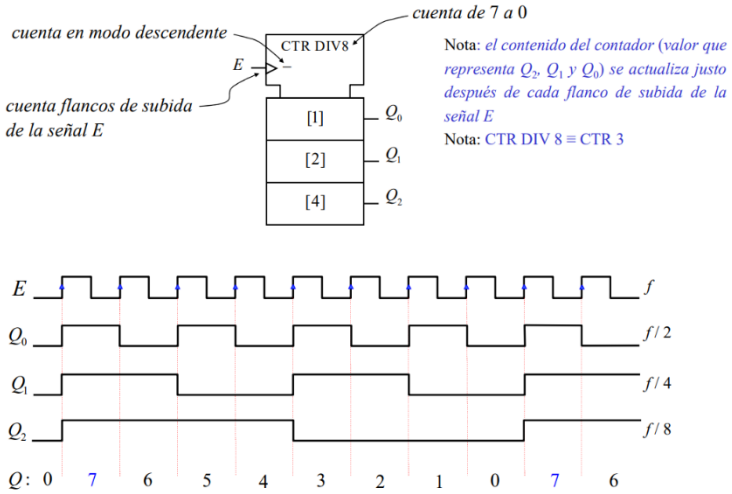
Ejemplo: contador de los flancos de bajada de la señal E, en modo ascendente (+).



Ejemplo: contador de los flancos de subida de la señal E, en modo descendente (–) de módulo 6.



Ejemplo: contador de los flancos de subida de la señal E, en modo descendente (–) de módulo 8.



Propiedad: Un contador es un circuito secuencial que genera una secuencia ordenada de números (≡ secuencia de contaje) de forma repetitiva.

Ejemplos: 0 - 1 - 2 - 3 - 4 - 5 - 6 - 7 - 8 - 9 - 0 - 1 - 2 - 3 - · · · (ascendente)

5 - 4 - 3 - 2 - 1 - 0 - 5 - 4 - 3 - · · · (descendente)

3 - 11 - 15 - 9 - 0 - 5 - 3 - 11 - 15 - · · · (cuenta en un código ≠ sistema binario)

Nota: en la literatura técnica se dice que "se produce un desbordamiento de un contador que cuenta en binario, en modo ascendente", cuando su contenido pasa del mayor valor de su secuencia de contaje al menor valor. Así, por ejemplo, cuando en un contador que cuenta en modo ascendente, con una secuencia de contaje: 0-1-2-3-4-5-6-7, su contenido pasa de 7 a 0 se dice que se ha producido un desbordamiento (the counter overflows) ≡ el contador no puede seguir contando flancos y reinicia el contaje.

Propiedad: Muchos contadores se diseñan haciendo que su salida coincida en todo momento con su estado interno.

Def.2: Se define el módulo de un contador como el número de valores que tiene su secuencia de contaje.

Ejemplo: 0 - 1 - 2 - 3 - 4 - 5 - 6 - 7 - 8 - 9 - 0 - 1 - 2 - 3 - · · · (módulo 10)

5 - 4 - 3 - 2 - 1 - 0 - 5 - 4 - 3 - · · · (módulo 6)

3 - 11 - 15 - 9 - 0 - 5 - 3 - 11 - 15 - · · · (módulo 6)

4 - 11 - 7 - 2 - 11 - 5 - 4 - 11 - 7 - 2 - · · · (módulo 6)* → en este caso, el contador no está diseñado de modo que su salida coincida con su estado interno.

Propiedad: Para guardar el estado interno de un contador se suelen utilizar flip-flops de tipo D o J-K, aunque se puede utilizar cualquier otro tipo de flip-flop.

Propiedad: En la práctica, los contadores se eligen en base a propiedades o características muy distintas entre sí:

$$
\text{Contadores}\begin{cases}
\text{Tipo de estructura}\begin{cases}\text{Asíncrona (obsoleta)}\\ \text{Síncrona}\begin{cases}\text{Basada en flip-flops y puertas}\\ \text{Basada en registros de desplazamiento}\end{cases}\end{cases}\\
\text{Secuencia de contaje}\begin{cases}\text{Números consecutivos}\begin{cases}\text{Modo ascendente}\\ \text{Modo descendente}\end{cases}\\ \text{Números no consecutivos}\end{cases}\\
\text{Código de la secuencia de contaje}\begin{cases}\text{_ Binario (natural)}\\ \text{_ BCD (natural)}\\ \text{_ Johnson}\\ \text{_ Anillo}\\ \text{_ otros}\end{cases}\\
\text{Módulo}\begin{cases}\text{Completo } (2^n \text{ estados con } n \text{ flip-flops})\\ \text{Incompleto } (< 2^n \text{ estados con } n \text{ flip-flops})\end{cases}
\end{cases}
$$

Contadores comerciales (cuentan en binario):

Notas sobre simbología:

DIV10 ≡ su módulo es igual a 10 (cuenta desde 0 hasta 9)

$\overline{R}$ ≡ *entrada de reset*: sirve para poner a 0 el contenido del contador
En este caso, es una entrada asíncrona, activa a nivel bajo.

M ≡ *entrada de selección del modo de funcionamiento*:

_ con $M = 0$ se selecciona el modo de "*carga en paralelo*"

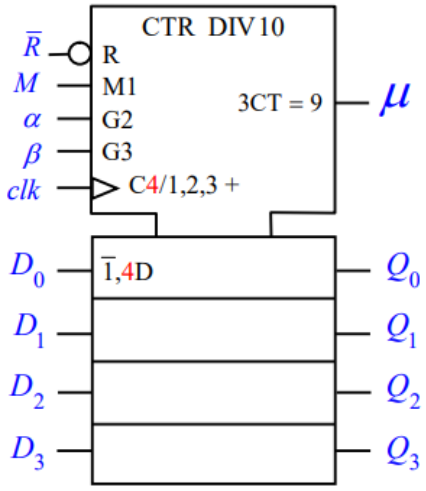
_ con $M = 1$ se selecciona el modo de "*contaje*"

α: *entrada de inhibición del contaje*

β: *entrada de inhibición del contaje y de la salida de fin de cuenta.*

clk (*entrada dinámica*): en este caso sincroniza tanto el contaje como la carga en paralelo (4).

μ≡ *salida que indica el fin de cuenta*: se pone a 1 cuando $\beta = 1$ y el contenido del contador es igual a 9_{10} ($Q_3,Q_2,Q_1,Q_0 = 1001_2$)

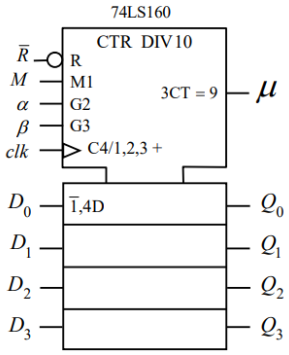


D_3,D_2,D_1,D_0: entradas en las que se aplica el valor a cargar en paralelo

Q_3,Q_2,Q_1,Q_0: en estas salidas se indica en todo momento el *contenido del contador en binario*

CT ≡ content ≡ contenido

Características del contador 74xxx160:

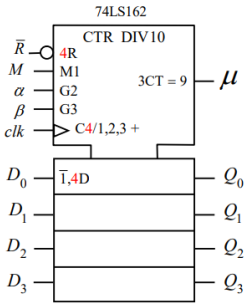


a) módulo: 10 (contador de décadas).
b) reset: asíncrono, activo a nivel bajo.
c) modo de contaje: ascendente.
d) secuencia de contaje: 0 – 1 – 2 – 3 – 4 – 5 – 6 – 7 – 8 – 9.
e) cuenta en binario.

Nota: en el momento en el que se activa la entrada de reset el contenido del contador se pone a cero (Q_3 ,Q_2 ,Q_1 ,Q_0 = 0000_2).

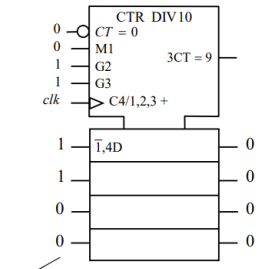
Importante: en los sistemas secuenciales la función de reset siempre es prioritaria frente a cualquier otra función (contaje, carga en paralelo, etc.). Esto es así por diseño (los sistemas secuenciales se diseñan para que esto sea siempre así).

Características del contador 74xxx162:

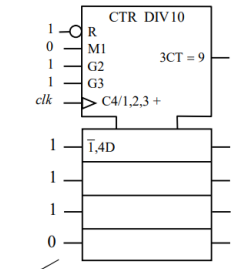


a) módulo: 10 (contador de décadas).
b) reset: síncrono, activo a nivel bajo
c) modo de contaje: ascendente
d) secuencia de contaje: 0 – 1 – 2 – 3 – 4 – 5 – 6 – 7 – 8 – 9
e) cuenta en binario

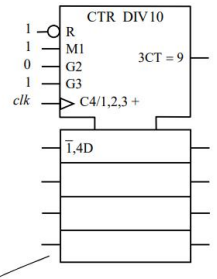
Nota: el contenido del contador se pone a cero si estando activa la entrada de reset, la señal clk describe un flanco de subida.



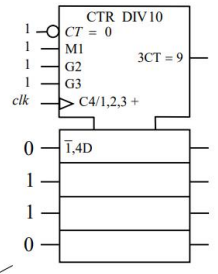
está seleccionado el modo de carga en paralelo, pero el contenido del contador está a cero porque la entrada de *reset* está activa.



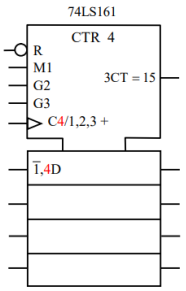
está seleccionado el modo de carga en paralelo. Cuando la señal *clk* describa un flanco de subida, el contenido del contador pasará a ser 0111_2.



Está seleccionado el modo de *contaje*, pero dicho modo de funcionamiento está *inhibido* (G2 = 0)



Está seleccionado el modo de *contaje*. Cada vez que la señal *clk* describa un flanco de *subida*, el contenido del contador se incrementará en una unidad. Cuando el contenido del contador valga 1001_2, con el siguiente flanco de subida de *clk*, el contenido del contador pasará a valer 0.

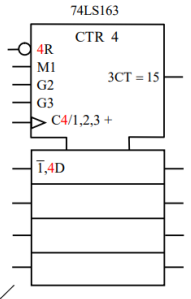


_ Contador de 4 bits (módulo igual a 16)

_ Reset *asíncrono*, activo a nivel bajo.

_ Cuenta desde 0 hasta 15

(CTR 4 ≡ CTR DIV 16)

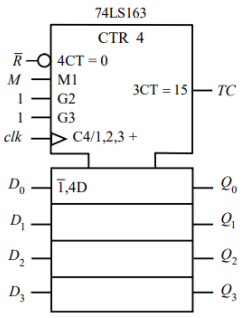


_ Contador de 4 bits (módulo igual a 16)

_ Reset *síncrono*, activo a nivel bajo.

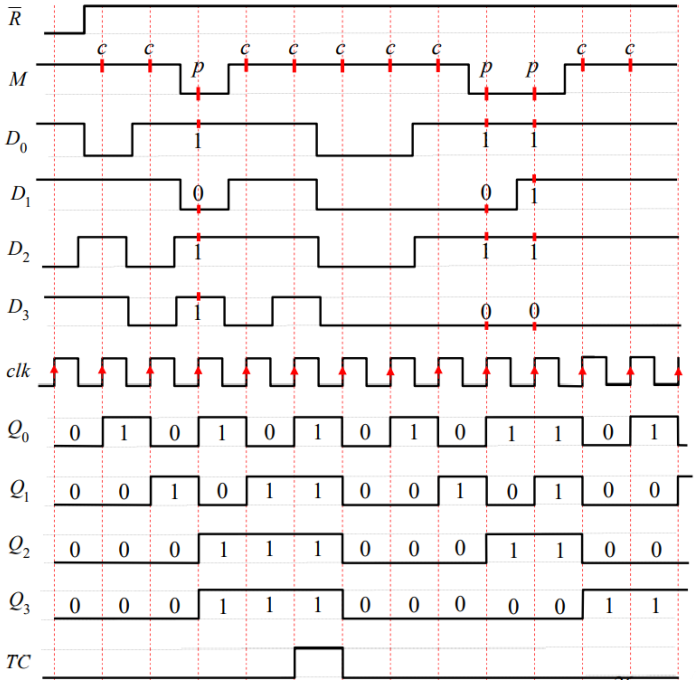
_ Cuenta desde 0 hasta 15

Ejemplo (cronograma):

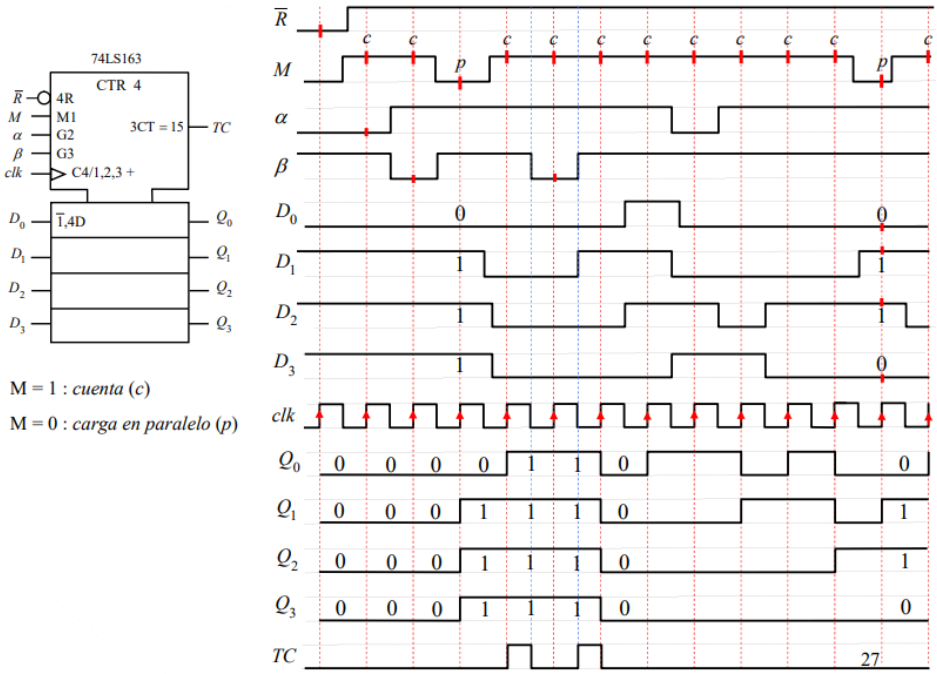


M = 1 : *cuenta* (*c*)

M = 0 : *carga en paralelo* (*p*)



Ejemplo (cronograma):



Registros

Definición y características:

Def.: Se entiende por registro de n bits a todo circuito capaz de guardar un dato de n bits.

a) Los registros se utilizan para guardar datos temporalmente.
b) Algunos tipos de registros se pueden utilizar para cambiar el formato de la información (serie ↔ paralelo).
c) Básicamente, un registro de n bits está formado por n flip-flops adecuadamente interconectados entre sí.
d) Los registros no tienen una secuencia de estados característica.
e) En un registro, los formatos de entrada y de salida de la información pueden ser de tipo:
    a. **Entrada serie:** si los bits a guardar en el registro se introducen uno a uno por una misma línea de entrada.
    b. **Entrada paralelo:** si los bits a guardar en el registro se introducen todos a la vez (se utiliza una línea independiente para cada bit).
    c. **Salida paralelo:** si los bits guardados en el registro se pueden leer (salen) todos a la vez (se utiliza una línea independiente para cada bit).
    d. **Salida serie:** si los bits guardados en el registro sólo se pueden leer (salen) de uno en uno, por una misma (y única) línea de salida.

f) De acuerdo con los formatos de entrada y de salida de la información, los registros se pueden clasificar de la siguiente manera:

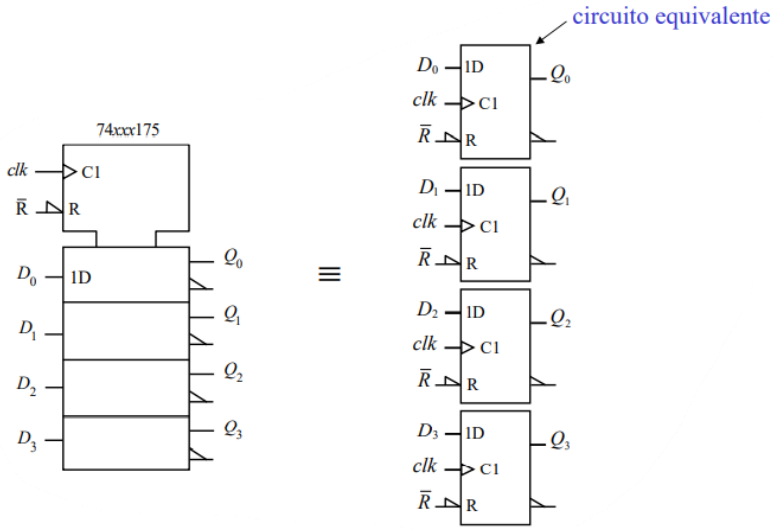
$$\text{Registros}\begin{cases}\text{Registros de almacenamiento : entrada paralelo y salida paralelo}\\ \text{Registros de desplazamiento}\begin{cases}\text{entrada serie y salida serie}\\ \text{entrada serie y salida paralelo}\\ \text{entrada paralelo y salida serie}\end{cases}\end{cases}$$

Importante: en los sistemas secuenciales la función de reset siempre es prioritaria frente a cualquier otra función (contaje, carga en paralelo, etc.). Esto es así por diseño (los sistemas secuenciales se diseñan para que esto sea así).

Registro de almacenamiento de n bits:

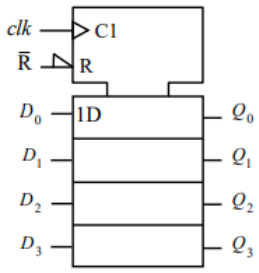
- Entrada: paralelo (los n bits se guardan a la vez).
- Salida: paralelo (se tiene acceso a los n bits guardados a la vez).

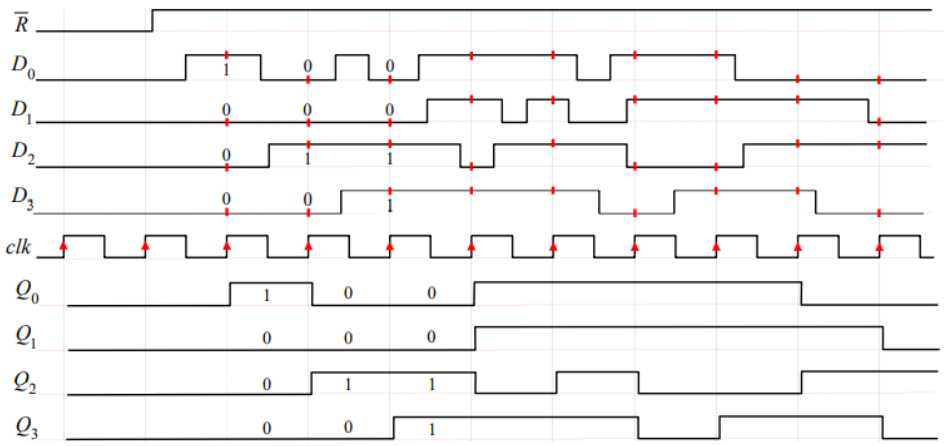
Ejemplo: registro paralelo de 4 bits.



Ejemplo (cronograma):

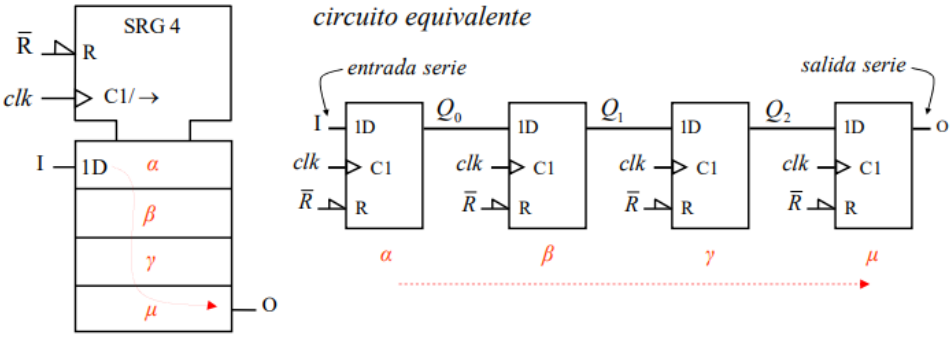
Nota: cada vez que la señal clk describe un flanco de subida, en el registro se guardan los valores que haya en las entradas D_3 , D_2 , D_1 y D_0 . A partir del momento en el que se activa la entrada de reset (R) en el registro se guarda el valor 0000. El contenido del registro se mantiene a cero (0000) mientras la entrada de reset esté activa. El reset es prioritario frente a la carga en paralelo.





Registro de desplazamiento (shift register) de n bits:

- Entrada: serie (los n bits se guardan de uno en uno).
- Salida: serie (se tiene acceso a los n bits guardados de uno en uno).

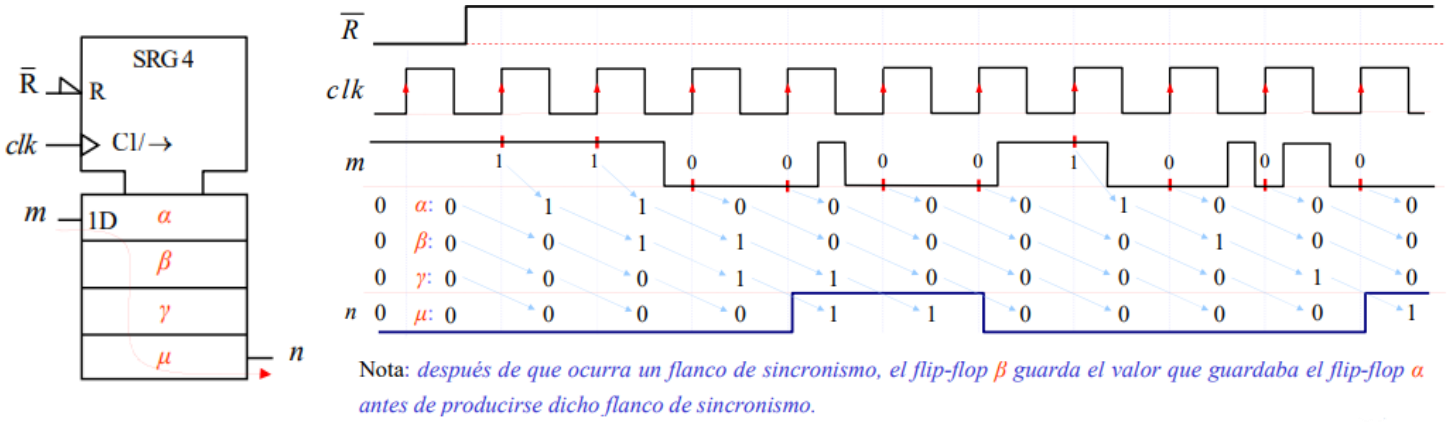


Importante: un flip-flop D guarda el valor que hay en su entrada de información (D) durante un flanco de sincronismo un poco después de que se produzca dicho flanco de sincronismo. Esto hace que, por ejemplo, después de que la señal clk describa un flanco de sincronismo el flip-flop μ guarde el valor que guardaba el flip-flop γ antes de producirse dicho flanco de sincronismo y que el flip-flop γ guarde el valor que guardaba el flip-flop β antes del flanco de sincronismo.

Ejemplo (cronograma):

Nota: con cada flanco de subida de la señal clk se produce un desplazamiento de los valores guardados en el registro, a la vez que se introduce (en el registro) el valor que haya en la entrada α. A partir del momento en el que se activa la entrada de reset (R), los valores de los 4 bits guardados en el registro pasan a ser 0000. El contenido del registro se mantiene a cero (0000) mientras la entrada de reset (R) esté activa.

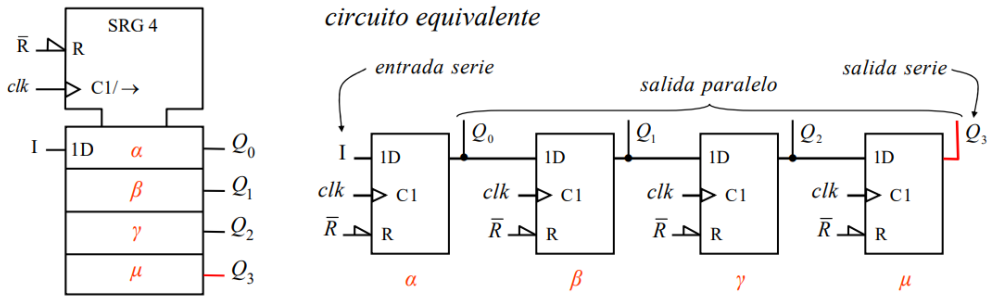
Importante: un flip-flop D guarda el valor que hay en su entrada D durante un flanco de sincronismo un poco un poco después de que se produzcan dicho flanco de sincronismo.



Nota: después de que ocurra un flanco de sincronismo, el flip-flop β guarda el valor que guardaba el flip-flop α antes de producirse dicho flanco de sincronismo.

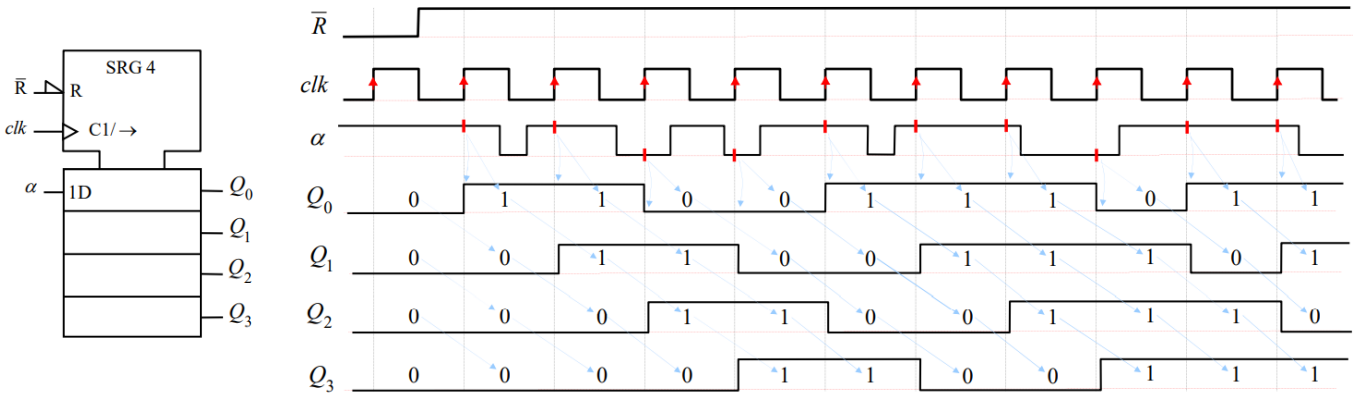
Registro de desplazamiento (shifter register) de n bits:

- Entrada: serie (los n bits se guardan de uno en uno).
- Salida: paralelo o serie (se tiene acceso a la vez a los n bits guardados).



Recuerda: un flip-flop D guarda un poco después de que se produzca un flanco de sincronismo el valor que hay en su entrada de información (D) durante el flanco de sincronismo. Esto hace que, por ejemplo, después de que la señal clk describa un flanco de sincronismo el flip-flop μ guarde el valor que guardaba el flip-flop γ antes de producirse dicho flanco de sincronismo y que el flip flop γ guarde el valor que guardaba el flip-flop β antes del flanco de sincronismo.

Ejemplo (cronograma):



Registros universales (universal register):

Pueden realizar cualquiera de las funciones analizadas anteriormente.

Existen unos registros denominados bidireccionales que disponen de dos entradas serie con las que se puede desplazar la información por su interior en ambos sentidos. Algunos registros universales son bidireccionales, otros no.

Aplicaciones de los registros:

- Guardar datos temporalmente.
- Cambiar el formato de la información (serie ↔paralelo).
- Implementación de contadores que cuentan en códigos especiales.
- etc.